

Actividad 6

Álvaro Del Valle Fernández

Índice

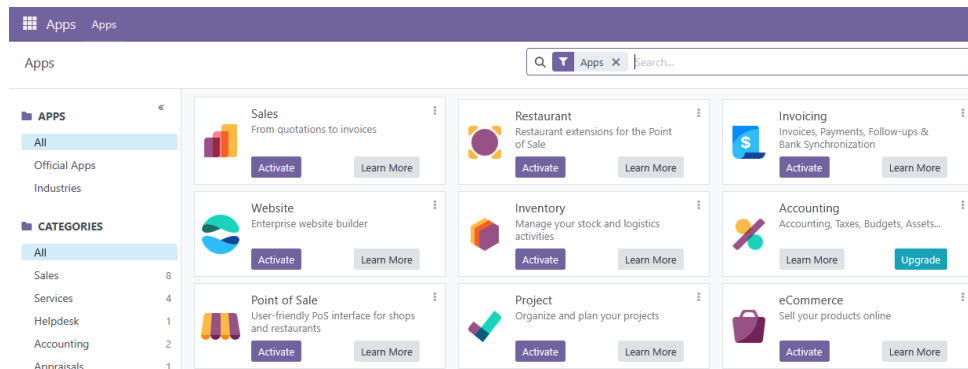
1. Introducción	4
2. Configuración base	5
2.1. init.py	5
2.2. manifest.py	6
2.3. docker-compose.yml	6
3. Models	7
3.1. Empleados	7
3.2. Vehículos	8
3.3. Clientes	9
3.4. Repartos	9
4. Vistas	10
4.1. Vista menu	10
4.2. Vista cliente	11
4.3. Vista empleado	13
4.4. Vista vehiculo	15
4.5. Vista reparto	17
5. Restricciones y validaciones	18
5.1. Empleado no tiene el carnet	18
5.2. Si el empleado o vehiculo ya estan en otro reparto:	18
5.3. Repartos de mas de 10km no se pueden realizar en bicicleta:	19
5.4. Repartos de menos de 1km no se pueden realizar en furgoneta:	19
5.5. Repartos ordenados por fecha de recepcion y urgencia:	19
5.6. Colores diferentes segun el estado del reparto:	19
6. Wizard	20
7. Informes	22
8. Controlador web	26
9. Ejemplos de uso del controlador web:	31
10.Security	33
11.Casos de uso:	33
11.1.Reparto empleado sin carnet adecuado:	33
11.2.Reparto empleado con carnet adecuado:	34
11.3.Reparto con vehiculo ocupado:	34
11.4.Reparto con empleado ya activo:	34
11.5.Reparto de mas de 10km en bicicleta:	34

11.6.Reparto de menos de 1km en furgoneta:	35
--	----

1. Introducción

Ahora necesitamos crear el nuevo modulo, para crearlo de forma mas sencilla usaré:

```
docker exec -it odoo18 odoo scaffold empresaReparto /mnt/extra-addons
```



Ahora creo los modelos que pide el ejercicio:

<code>_init_.py</code>	2/18/2026 12:25 PM	Python Script	1 KB
<code>cliente.py</code>	2/18/2026 12:28 PM	Python Script	0 KB
<code>empleados.py</code>	2/18/2026 12:27 PM	Python Script	0 KB
<code>models.py</code>	2/18/2026 12:25 PM	Python Script	1 KB
<code>repartos.py</code>	2/18/2026 12:29 PM	Python Script	0 KB
<code>vehiculos.py</code>	2/18/2026 12:27 PM	Python Script	0 KB

```
cd C:\Users\delva\Desktop\Practica6
docker compose up --build
```

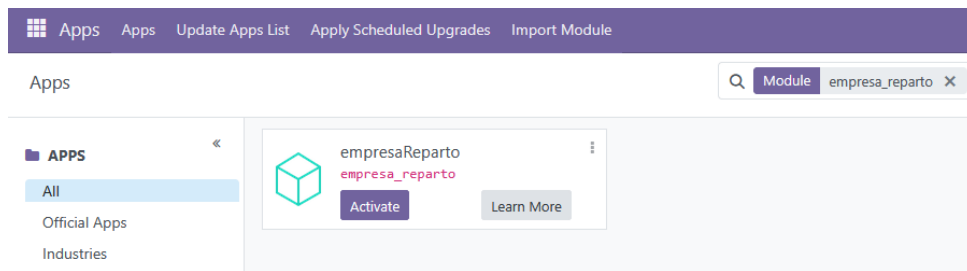
Tras multiples horas intenando hacer que funcione odoo creando el modulo desde 0, Odoo no es capaz de enviar ningun codigo de error ni indicar por que el modulo se descarga pero no funciona. Debido a esto uso la base del ejericio 07 de futbol para no perder aun mas tiempo.

Tras horas reestructurando el trabajo el modulo nuevo de Futbol adaptado no funciona, al no permitir ni desinstalarlo, no dando error siquiera.

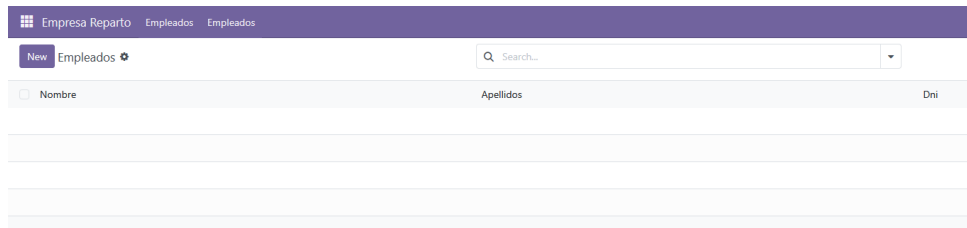
Como la version adaptada funcionaba incluso peor decidi volver de nuevo a la primera version, perdiendo todas las horas de trabajo en adaptar la version 07.

Esta version no funciona pero por lo menos puedo borrar el modulo y odoo es capaz de indicar algun error aunque nunca de forma precisa.

La base de datos funciona y el modulo se carga, pero los modulos y el wizard dan errores de forma interrumpida, perdiendo horas.



Tras varias horas de errores entre Odoo y los log de docker conseguí la primera version estable:



La version es estable, aunque añañi solo los elemntos basicos junto el primer botón, ahora debo añadir todas las vistas adecuadas.

```
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
  <menuitem id="menu_empresa_reparto_root"
    name="Empresa de Reparto"/>

  <record id="action_empresa_reparto_empleados" model="ir.actions.act_window">
    <field name="name">Empleados</field>
    <field name="res_model">reparto.empleado</field>
    <field name="view_mode">list,form</field>
  </record>

  <menuitem id="menu_empresa_reparto_empleados"
    name="Empleados"
    parent="menu_empresa_reparto_root"
    action="action_empresa_reparto_empleados"/>
</odoo>
```

Elimino el test de boton de empleados dentro de empeladosviews.xml para evitar el duplicado.

2. Configuración base

En este apartado simplemente mostraré los archivos configuración base y de carga de datos.

2.1. init.py

Indica las rutas de los archivos que deben cargarse:

```
# -*- coding: utf-8 -*-
```

```
from . import models
from . import controllers
from . import wizard
```

2.2. manifest.py

Elemento de configuracion principal, se encarga de identificar y cargar los elementos, simplemente indica dependencias, muestra las rutas de datos a cargar y que se muestre como aplicacion descargable:

```
# -*- coding: utf-8 -*-
{
    'name': 'Empresa de Repartos',
    'version': '1.0',
    'description': 'Gestion de repartos',
    'author': 'Tu Nombre',
    'depends': ['base'],
    'data': [
        'security/ir.model.access.csv',
        'data/sequence_data.xml',
        'views/empleado_views.xml',
        'views/vehiculo_views.xml',
        'views/cliente_views.xml',
        'views/reparto_views.xml',
        'views/menu.xml',
        'reports/reparto_report.xml',
    ],
    'demo': [],
    'installable': True,
    'application': True,
    'license': 'LGPL-3',
}
```

2.3. docker-compose.yml

Configuración de docker, configura el contenedor de odoo y el de la base de datos indicando los puertos y rutas. Eliminé el campo de version porque daba error.

```
services:
  db:
    image: postgres:15
    container_name: odoo18-db
```

```

environment:
  POSTGRES_DB: postgres
  POSTGRES_USER: odoo
  POSTGRES_PASSWORD: odoo
volumes:
  - odoo18-db-data:/var/lib/postgresql/data

odoo:
  image: odoo:18
  container_name: odoo18
  depends_on:
    - db
  ports:
    - "8069:8069"
  volumes:
    - ./addons:/mnt/extra-addons
    - ./config/odoo.conf:/etc/odoo/odoo.conf
    - odoo18-web-data:/var/lib/odoo
  command: odoo -c /etc/odoo/odoo.conf

volumes:
  odoo18-db-data:
  odoo18-web-data:

```

3. Models

Dentro de este apartado mostraré el código de cada uno de los modelos creados, todos siguiendo las indicaciones del ejercicio.

3.1. Empleados

```

from odoo import models, fields, api

class Empleado(models.Model):
    _name = 'reparto.empleado'
    _description = 'Empleado'

    nombre = fields.Char(required=True)
    apellidos = fields.Char(required=True)
    dni = fields.Char(required=True)
    telefono = fields.Char()
    foto = fields.Binary()

```

```

carnet_ciclomotor = fields.Boolean()
carnet_furgoneta = fields.Boolean()

reparto_ids = fields.One2many('reparto.reparto', '
    empleado_id')

def open_repartos_pendientes(self):
    """Abre el informe de repartos"""
    self.ensure_one()

    action = self.env.ref('empresa.reparto.
        action_report_reparto', raise_if_not_found=
        False)
    if not action:

        return {
            'type': 'ir.actions.client',
            'tag': 'display_notification',
            'params': {
                'title': 'Error',
                'message': 'No se encontro la accion
                    del informe',
                'type': 'danger',
                'sticky': True,
            }
        }
    return action.report_action(self)

```

3.2. Vehículos

```

from odoo import models, fields, api

class Vehiculo(models.Model):
    _name = 'reparto.vehiculo'

    tipo = fields.Selection([
        ('bicicleta', 'Bicicleta'),
        ('furgoneta', 'Furgoneta')
    ], required=True)

    matricula = fields.Char()
    foto = fields.Binary()

```



```

descripcion = fields.Text()

estado = fields.Selection([
    ('disponible', 'Disponible'),
    ('reparto', 'En reparto')
], default='disponible')

```

3.3. Clientes

```

from odoo import models, fields, api

class Cliente(models.Model):
    _name = 'reparto.cliente'

    dni = fields.Char(required=True)
    nombre = fields.Char(required=True)
    apellidos = fields.Char()
    telefono = fields.Char()

```

3.4. Repartos

Este es el modelo mas complejo, cumpliendo todas las condiciones del ejercicio. Las restricciones las mostraré en el apartado de restricciones y validaciones, aqui solo muestro el modelo sin restricciones.

```

from odoo import models, fields, api
from odoo.exceptions import ValidationError

class Reparto(models.Model):
    _name = 'reparto.reparto'
    _description = 'Reparto'
    _order = 'fecha_recepcion desc, urgencia'

    codigo = fields.Char(default='Nuevo', readonly=True)
    fecha_inicio = fields.Datetime()
    fecha_retorno = fields.Datetime()
    fecha_recepcion = fields.Datetime(required=True)

    empleado_id = fields.Many2one('reparto.empleado',
                                   required=True)
    vehiculo_id = fields.Many2one('reparto.vehiculo',
                                   required=True)

```

```

kilometros = fields.Float()
peso = fields.Float()
volumen = fields.Float()
observaciones = fields.Text()

estado = fields.Selection([
    ('no_salido', 'No ha salido'),
    ('camino', 'En camino'),
    ('entregado', 'Entregado')
], default='no_salido')

urgencia = fields.Selection([
    ('organos', 'Organos humanos'),
    ('refrigerado', 'Alimentos refrigerados'),
    ('alimentos', 'Alimentos'),
    ('alta', 'Alta prioridad'),
    ('baja', 'Baja prioridad')
])

cliente_id = fields.Many2one('reparto.cliente')
receptor = fields.Char()

```

4. Vistas

4.1. Vista menu

Esta es la vista del menu, simplemente muestra las opciones para cada modelo, indicando el modelo a mostrar y el modo de vista:

```

<?xml version="1.0" encoding="UTF-8"?>
<odoo>
    <menuitem id="menu_root" name="Empresa de Reparto"/>

    <record id="action_empleados_view" model="ir.actions.act_window">
        <field name="name">Empleados</field>
        <field name="res_model">reparto.empleado</field>
        <field name="view_mode">list,form</field>
    </record>
    <menuitem id="menu_empresa_reparto_empleados"
        name="Empleados"
        parent="menu_root"
        action="action_empleados_view"/>

```

```

<record id="action_vehiculos_view" model="ir.actions.act_window">
    <field name="name">Vehiculos</field>
    <field name="res_model">reparto.vehiculo</field>
    <field name="view_mode">list,form</field>
</record>
<menuitem id="menu_empresa_reparto_vehiculos"
    name="Vehiculos"
    parent="menu_root"
    action="action_vehiculos_view"/>

<record id="action_clientes_view" model="ir.actions.act_window">
    <field name="name">Clientes</field>
    <field name="res_model">reparto.cliente</field>
    <field name="view_mode">list,form</field>
</record>
<menuitem id="menu_empresa_reparto_clientes"
    name="Clientes"
    parent="menu_root"
    action="action_clientes_view"/>

<record id="action_repartos_view" model="ir.actions.act_window">
    <field name="name">Repartos</field>
    <field name="res_model">reparto.reparto</field>
    <field name="view_mode">list,form</field>
</record>
<menuitem id="menu_empresa_reparto_repartos"
    name="Repartos"
    parent="menu_root"
    action="action_repartos_view"/>

</odoo>

```

4.2. Vista cliente

Aqui se muestra la vista de cliente, con los campos mas importantes, el resto de campos se pueden ver en el formulario:

Empresa de Reparto Empleados Vehiculos Clientes Repartos			
New Clientes		Search...	
☐ Dni	Nombre	Apellidos	Telefono
☐ 38764862E	Iria	Suarez	32152342467
☐ 83764238648A	Ivan	Smith	1283721897



New

Cientes

reparto.cliente,2 

Dni 38764862E

Nombre Iria

Apellidos Suarez

Telefono 32152342467

```
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
  <record id="view_cliente_list" model="ir.ui.view">
    <field name="name">reparto.cliente.list </field>
    <field name="model">reparto.cliente </field>
    <field name="arch" type="xml">
      <list>
        <field name="dni"/>
        <field name="nombre"/>
        <field name="apellidos"/>
        <field name="telefono"/>
      </list>
    </field>
  </record>
```

```

<record id="view_cliente_form" model="ir.ui.view">
  <field name="name">reparto.cliente.form</field>
  <field name="model">reparto.cliente</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <group>
          <field name="dni"/>
          <field name="nombre"/>
          <field name="apellidos"/>
          <field name="telefono"/>
        </group>
      </sheet>
    </form>
  </field>
</record>
</odoo>

```

4.3. Vista empleado

En esta vista podemos ver la informacion mas importante de cada empleado.

Empresa de Reparto Empleados Vehículos Clientes Repartos			
New Empleados		Q search...	
☐ Nombre	Apellidos	Dni	Telefono
☐ Ramon	Perez	895763956E	347657896238
☐ Jose	Lerez	345354364E	45364535

[New](#)
[Empleados](#)
[reparto.empleado,1](#)


Nombre	Ramon
Apellidos	Perez
Dni	895763956E
Telefono	347657896238
Foto	111.64 Kb
Carnet Ciclomotor	☒
Carnet Furgoneta	☒

[New](#)
[Empleados](#)
[reparto.empleado,1](#)


[Repartos Pendientes](#)

Nombre	Ramon	Carnet Ciclomotor	☒
Apellidos	Perez	Carnet Furgoneta	☒
Dni	895763956E	Foto	
Telefono	347657896238		

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<odoo>
```

```
<!-- Vista lista -->
```

```
<record id="view_empleado_list" model="ir.ui.view">
```

```
<field name="name">empleado.list </field>
```

```
<field name="model">reparto.empleado</field>
```

```
<field name="arch" type="xml">
```

```
<list>
```

```
<field name="nombre"/>
```

```
<field name="apellidos"/>
```

```
<field name="dni"/>
```

```
<field name="telefono"/>
```

```
</list>
```




New

Vehículos

reparto.vehiculo,1 

Tipo	Bicicleta
Matricula	H21JK321B
Descripcion	Bicicleta con motor a dos tiempos
Estado	Disponibile

```
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
  <record id="view_vehiculo_list" model="ir.ui.view">
    <field name="name">reparto.vehiculo.list </field>
    <field name="model">reparto.vehiculo </field>
    <field name="arch" type="xml">
      <list>
        <field name="tipo"/>
        <field name="matricula"/>
        <field name="estado"/>
      </list>
    </field>
  </record>

  <record id="view_vehiculo_form" model="ir.ui.view">
    <field name="name">reparto.vehiculo.form </field>
    <field name="model">reparto.vehiculo </field>
    <field name="arch" type="xml">
      <form>
        <sheet>
          <group>
```



```

        <field name="tipo"/>
        <field name="matricula"/>
        <field name="descripcion"/>
        <field name="estado"/>
    </group>
</sheet>
</form>
</field>
</record>
</odoo>

```

4.5. Vista reparto

Al utilizar el wizard para crear los repartos, la vista de lista y formulario de reparto es muy simple, mostrando solo los campos basicos, el resto de campos se rellenan en el wizard. Dentro del apartado de wizard mostraré esa vista de formulario.

The top screenshot shows a list view of 'Repartos' (Deliveries) in the 'Empresa de Reparto' (Delivery Company) context. The table has columns: Empleado, Vehículo, Cliente, and Fecha Recepcion. It lists two records: 'reparto.empleado.1' with vehicle 'reparto.vehiculo.2' and date '02/26/2026 04:00:00', and 'reparto.empleado.2' with vehicle 'reparto.vehiculo.1' and date '02/25/2026 04:00:00'.

The bottom screenshot shows the 'New' form for 'Repartos'. It contains the following fields and values:

- Codigo: Nuevo
- Fecha Retorno: 02/27/2026 04:00:00
- Empleado: reparto.empleado.1
- Kilometros: 9.00
- Volumen: 30.00
- Observaciones: Reparto en Furgoneta
- Estado: En camino
- Cliente: reparto.cliente.2

Summary section:

- Fecha Inicio: 02/20/2026 03:00:00
- Fecha Recepcion: 02/26/2026 04:00:00
- Vehiculo: reparto.vehiculo.2
- Peso: 20.00
- Urgencia: Baja prioridad
- Receptor:

```

<?xml version="1.0" encoding="UTF-8"?>
<odoo>
    <record id="view_reparto_list" model="ir.ui.view">
        <field name="name">reparto.list</field>
        <field name="model">reparto.reparto</field>
        <field name="arch" type="xml">
            <list>
                <field name="empleado_id"/>
                <field name="vehiculo_id"/>
                <field name="cliente_id"/>
                <field name="fecha_recepcion"/>
            </list>
        </field>
    </record>
</odoo>

```

```

        </list>
    </field>
</record>

<record id="action_repartos_view" model="ir.actions.act_window">
    <field name="name">Repartos</field>
    <field name="res_model">reparto.reparto</field>
    <field name="view_mode">list,form</field>
    <field name="context">{'default_open_wizard':
        True}</field>
</record>
</odoo>

```

5. Restricciones y validaciones

Ahora mostraré mas restricciones pedidas en el enunciado

5.1. Empleado no tiene el carnet

Codigo para validar:

```

@api.constrains('empleado_id', 'vehiculo_id')
def _check_carnet(self):
    for record in self:
        if record.vehiculo_id.tipo == 'bicicleta' and not record.
            empleado_id.carnet_ciclomotor:
            raise ValidationError("El empleado no tiene carnet de
                ciclomotor.")
        if record.vehiculo_id.tipo == 'furgoneta' and not record.
            empleado_id.carnet_furgoneta:
            raise ValidationError("El empleado no tiene carnet de
                furgoneta.")

```

5.2. Si el empleado o vehiculo ya estan en otro reparto:

```

@api.constrains('empleado_id', 'estado')
def _check_reparto_activo(self):
    for record in self:
        activos = self.search([
            ('empleado_id', '=', record.empleado_id.id),
            ('estado', '=', 'camino'),

```

```

        ('id', '!=', record.id)
    ])
    if activos and record.estado == 'camino':
        raise ValidationError("El empleado ya tiene un
                               reparto activo.")

```

5.3. Repartos de mas de 10km no se pueden realizar en bicicleta:

Codigo para validar:

```

@api.constrains('kilometros', 'vehiculo_id')
def _check_km(self):
    for record in self:
        if record.vehiculo_id.tipo == 'bicicleta' and record.
           kilometros > 10:
            raise ValidationError("Mas de 10km no puede hacerse
                                   en bicicleta.")

```

5.4. Repartos de menos de 1km no se pueden realizar en furgoneta:

```

    if record.vehiculo_id.tipo == 'furgoneta' and record.
       kilometros < 1:
        raise ValidationError("Menos de 1km no puede hacerse
                               en furgoneta.")

```

5.5. Repartos ordenados por fecha de recepcion y urgencia:

Simplemente con el campo order los repartos se ordenan correctamente:

```

_order = 'fecha_recepcion desc, urgencia '

```

5.6. Colores diferentes segun el estado del reparto:

El atributo decoración cambia el color del texto segun el estado:

```

<list
    decoration-danger="estado == 'camino'"
    decoration-success="estado == 'entregado'"
    decoration-muted="estado == 'no_salido'">

```

Fecha Recepcion

02/26/2026 04:00:00

02/25/2026 04:00:00

6. Wizard

Wizard para repartos, al hacer click en New se abre el wizard para rellenar los campos necesarios, ahora mostraré los documentos. También realiza un check por si no cumple las restricciones definidas en el modelo reparto.

Primero el reparto wizard.py:

```
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class RepartoWizard(models.TransientModel):
    _name = 'reparto.reparto_wizard'

    empleado_id = fields.Many2one('reparto.empleado', string="Empleado", required=True)
    vehiculo_id = fields.Many2one('reparto.vehiculo', string="Vehiculo", required=True)
    cliente_id = fields.Many2one('reparto.cliente', string="Cliente", required=True)
    def crear_reparto(self):
        Reparto = self.env['reparto.reparto']

#Crear el reparto
        reparto = Reparto.new({
```

```

        'empleado_id': self.empleado_id.id,
        'vehiculo_id': self.vehiculo_id.id,
        'cliente_id': self.cliente_id.id,
        'fecha_recepcion': fields.Datetime.now(),
        'estado': 'camino',
    })

    reparto._check_carnet()
    reparto._check_reparto_activo()
    reparto._check_km()
    reparto_id = Reparto.create(reparto._convert_to_write
                                (reparto._cache))
    return {'type': 'ir.actions.act_window_close'}

```

Ahora el reparto wizard xml que se encarga de la estructura de la vista del wizard:

```

<!-- Vista -->
<record id="view_reparto_wizard_form" model="ir.ui.view">
    <field name="name">reparto.wizard.form</field>
    <field name="model">reparto.reparto_wizard</field>
    <field name="arch" type="xml">
        <form string="Crear Reparto">
            <group>
                <field name="empleado_id"/>
                <field name="vehiculo_id"/>
                <field name="cliente_id"/>
            </group>
            <footer>
                <button string="Crear" type="object" name
                    ="crear_reparto" class="btn-primary"/>
                <button string="Cancelar" class="btn-
                    secondary" special="cancel"/>
            </footer>
        </form>
    </field>
</record>
<!-- Abrir -->
<record id="action_reparto_wizard" model="ir.actions.
    act_window">
    <field name="name">Crear Reparto</field>
    <field name="res_model">reparto.reparto_wizard</field>
    <field name="view_mode">form</field>
    <field name="target">new</field>

```

```
</record>
</odoo>
```

7. Informes

Informe para cada empleado con los repartos pendientes, este elemento se situa en reports, reparto report. xml y es activado mediante el boton de la vista de empleado, el cual abre el informe.

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <template id="report_repartos_pendientes_empleado">
    <t t-call="web.html_container">
      <t t-foreach="docs" t-as="empleado">
        <t t-call="web.external_layout">
          <div class="page">
            <div class="oe_structure"/>
            <div class="row">
              <div class="col-xs-12">
                <h2>Repartos Pendientes - <t
                  t-esc="empleado.nombre"/>
                <t t-esc="empleado.
                  apellidos"/></h2>
                <p><strong>DNI:</strong> <t t
                  -esc="empleado.dni"/></p>
              </div>
            </div>
            <div class="row">
              <div class="col-xs-12">
                <table class="table table-
                  bordered">
                  <thead>
                    <tr>
                      <th>Codigo</th>
                      <th>Cliente</th>
                      <th>Vehiculo</th>
                      <th>Fecha
                        recepcion</th>
                      <th>Urgencia</th>
                    </tr>
                  </thead>
```

```

<tbody>
  <t t-set="
    repartos_pendientes
    " t-value="
    empleado.
    reparto_ids.
    filtered(lambda r:
      r.estado == '
      no_salido')"/>
  <t t-foreach="
    repartos_pendientes
    " t-as="r">
    <tr>
      <td><t t-esc
        ="r.codigo
        "/></td>
      <td><t t-esc
        ="r.
        cliente_id
        .nombre or
        ''"/> <t
        t-esc="r.
        cliente_id
        .apellidos
        or
        ''"/></td>
      <td><t t-esc
        ="r.
        vehiculo_id
        .tipo or
        ''"/></td>
      <td><t t-esc
        ="r.
        fecha_recepcion
        "/></td>
      <td><t t-esc
        ="r.
        urgencia
        "/></td>
    </tr>
  </t>
  <t t-if="not

```

```

repartos_pendientes
">
    <tr>
        <td colspan
            ="5" style
            ="text-align:
            center;">
            No hay
            repartos
            pendientes
        </td>
    </tr>
</tbody>
</table>
</div>
</div>
</div>
</t>
</t>
</template>

<record id="action_report_reparto" model="ir.actions.
report">
    <field name="name">Repartos pendientes</field>
    <field name="model">reparto.empleado</field>
    <field name="report_type">qweb-pdf</field>
    <field name="report_name">empresa_reparto.
        report_repartos_pendientes_empleado</field>
    <field name="report_file">empresa_reparto.
        report_repartos_pendientes_empleado</field>
    <field name="binding_model_id" ref="
        model_reparto_empleado"/>
    <field name="binding_type">report</field>
    <field name="print_report_name">'Repartos Pendientes.
        %s' % (object.nombre)</field>
</record>
</odoo>

```

Archivo Reparto. py:

```
from odoo import models
```



```

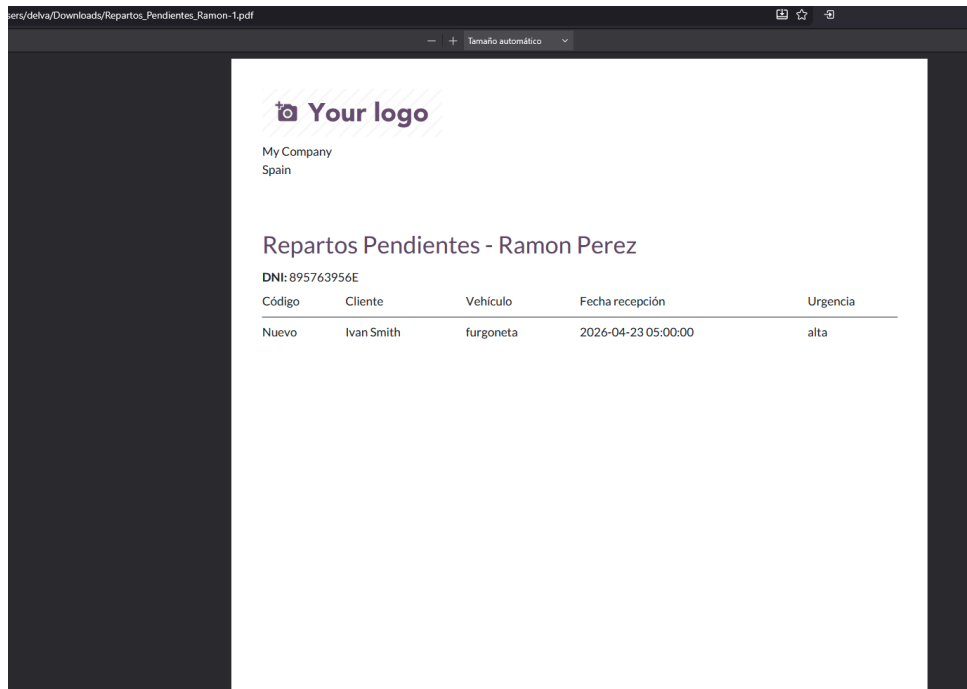
class ReportRepartoPendientes(models.AbstractModel):
    _name = 'report.empresa_reparto.'
    report_repartos_pendientes_empleado '
    _description = 'Reporte de Repartos Pendientes '

    def _get_report_values(self, docids, data=None):
        docs = self.env['reparto.empleado'].browse(docids)
        return {
            'doc_ids': docids,
            'doc_model': 'reparto.empleado',
            'docs': docs,
        }

```

Aqui podemos ver un reporte creado correctamente:

The screenshot shows a 'Configure your document layout' window. On the left, there are configuration options for the report layout, including Layout (Light, Boxed, Bold, Striped, Bubble, Wave, Folder), Background (Blank), Text (Lato), Logo (Your logo), Colors (two color swatches), Address (My Company Spain), Tagline (e.g. Global Business Solutions), Footer (admin@test.com), and Paper format (A4). On the right, a preview of the generated report is shown. The report is an invoice titled 'Invoice INV/2023/00003' with an invoice date of 07/08/2020 and a due date of 08/07/2020. It includes a table with columns for Description, Quantity, Unit Price, Taxes, and Amount. The table lists two items: 'Three Seater Sofa with Lounger in Steel Grey Colour' and 'Four person modern office workstation'. The total amount is \$22,137.50. The report also includes a company logo, address, and contact information.



8. Controlador web

Controlador web, esta es la version estable del controlador, no encontré una forma de que no me diese error con la id del reparto. Esta se compone de tres endpoints, endpoint que obtiene un JSON mediante el codigo, endpoint JSON listado y el endpoint para el html el cual permite mostrar la interfaz:

```
from odoo import http
from odoo.http import request
import json

class RepartoController(http.Controller):

    @http.route('/repartos/estado/<string:codigo_reparto>', type='
    http', auth='public', methods=['GET'], csrf=False)
    def obtener_estado_reparto(self, codigo_reparto, **kwargs):
        try:
            reparto = request.env['reparto.reparto'].sudo().search([
                ('codigo', '=', codigo_reparto)
            ], limit=1)
#test error de id
            if not reparto and codigo_reparto.isdigit():
                reparto = request.env['reparto.reparto'].sudo().
                    browse(int(codigo_reparto))

            if not reparto:
```

```

return request.make_response(
    json.dumps({
        'success': False,
        'error': f'No se encontro el reparto con
            codigo: {codigo_reparto}'
    }, ensure_ascii=False),
    headers=[('Content-Type', 'application/json')]
)

```

#Obtener la info

```

estado_texto = dict(reparto._fields['estado'].selection).
    get(reparto.estado)
urgencia_texto = dict(reparto._fields['urgencia'].
    selection).get(reparto.urgencia)

```

```

resultado = {
    'success': True,
    'data': {
        'codigo': reparto.codigo,
        'id': reparto.id,
        'estado': reparto.estado,
        'estado_descripcion': estado_texto,
        'urgencia': reparto.urgencia,
        'urgencia_descripcion': urgencia_texto,
        'fecha_recepcion': reparto.fecha_recepcion.
            strftime('%d/%m/%Y %H:%M') if reparto.
            fecha_recepcion else None,
        'fecha_inicio': reparto.fecha_inicio.strftime('%d
            /%m/%Y %H:%M') if reparto.fecha_inicio else
            None,
        'fecha_retorno': reparto.fecha_retorno.strftime
            ('%d/%m/%Y %H:%M') if reparto.fecha_retorno
            else None,
        'repartidor': {
            'id': reparto.empleado_id.id,
            'nombre': reparto.empleado_id.nombre,
            'apellidos': reparto.empleado_id.apellidos,
            'dni': reparto.empleado_id.dni
        } if reparto.empleado_id else None,
        'vehiculo': {
            'id': reparto.vehiculo_id.id,
            'tipo': reparto.vehiculo_id.tipo,

```

```

        'descripcion': reparto.vehiculo_id.descripcion,
        'matricula': reparto.vehiculo_id.matricula
    } if reparto.vehiculo_id else None,
    'cliente_emisor': {
        'id': reparto.cliente_id.id,
        'nombre': reparto.cliente_id.nombre,
        'apellidos': reparto.cliente_id.apellidos,
        'dni': reparto.cliente_id.dni
    } if reparto.cliente_id else None,
    'receptor': reparto.receptor,
    'kilometros': reparto.kilometros,
    'peso': reparto.peso,
    'volumen': reparto.volumen,
    'observaciones': reparto.observaciones
    }
}

return request.make_response(
    json.dumps(resultado, ensure_ascii=False),
    headers=[('Content-Type', 'application/json')]
)

except Exception as e:
    return request.make_response(
        json.dumps({
            'success': False,
            'error': str(e)
        }, ensure_ascii=False),
        headers=[('Content-Type', 'application/json')]
    )

@http.route('/repartos/estado', type='http', auth='public',
    methods=['GET'], csrf=False)
def listar_repartos(self, **kwargs):
    try:
        repartos = request.env['reparto.reparto'].sudo().search(
            [], limit=100)

        resultado = {
            'success': True,
            'total': len(repartos),

```

```

        'data': []
    }

    for reparto in repartos:
        estado_texto = dict(reparto._fields['estado'].
            selection).get(reparto.estado)
        resultado['data'].append({
            'codigo': reparto.codigo,
            'id': reparto.id,
            'estado': reparto.estado,
            'estado_descripcion': estado_texto,
            'fecha_recepcion': reparto.fecha_recepcion.
                strftime('%d/%m/%Y %H:%M') if reparto.
                fecha_recepcion else None,
            'repartidor': reparto.empleado_id.nombre if
                reparto.empleado_id else None,
            'urgencia': reparto.urgencia
        })

    return request.make_response(
        json.dumps(resultado, ensure_ascii=False),
        headers=[('Content-Type', 'application/json')]
    )

except Exception as e:
    return request.make_response(
        json.dumps({
            'success': False,
            'error': str(e)
        }, ensure_ascii=False),
        headers=[('Content-Type', 'application/json')]
    )

@http.route('/repartos/web/estado/<string:codigo_reparto>', type
    ='http', auth='public', methods=['GET'], csrf=False)
def web_estado_reparto(self, codigo_reparto, **kwargs):
    reparto = request.env['reparto.reparto'].sudo().search([
        ('codigo', '=', codigo_reparto)
    ], limit=1)

    if not reparto and codigo_reparto.isdigit():
        reparto = request.env['reparto.reparto'].sudo().browse(

```

```

        int(codigo_reparto))

if not reparto:
    return """
    <html>
        <head><title>Reparto no encontrado</title></head>
        <body>
            <h1>Error</h1>
            <p>No se encontro el reparto con codigo: {}</p>
        </body>
    </html>
    """.format(codigo_reparto)

#test, no funciona
color_estado = {
    'no_salido': 'gray',
    'camino': 'orange',
    'entregado': 'green'
}.get(reparto.estado, 'black')

estado_texto = dict(reparto._fields['estado'].selection).get(
    reparto.estado)
urgencia_texto = dict(reparto._fields['urgencia'].selection).
    get(reparto.urgencia)

return """
<html>
    <head>
        <title>Estado del Reparto {}</title>
        <style>
            body {{ font-family: Arial; margin: 40px; }}
            .card {{ border: 1px solid #ddd; padding: 20px;
                border-radius: 5px; }}
            .estado {{ color: {}; font-weight: bold; }}
            .label {{ font-weight: bold; color: #666; }}
        </style>
    </head>
    <body>
        <h1>Estado del Reparto {}</h1>
        <div class="card">
            <p><span class="label">Codigo:</span> {}</p>
            <p><span class="label">Estado:</span> <span class
                ="estado">{}</span></p>

```

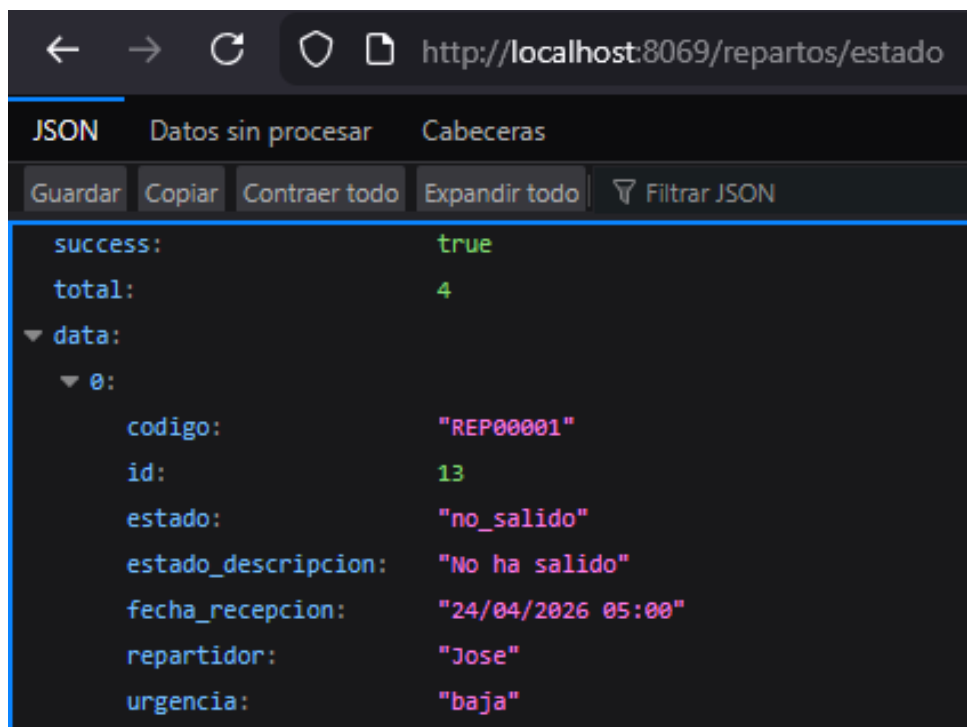
```

<p><span class="label">Urgencia:</span> {}</p>
<p><span class="label">Fecha recepcion:</span>
    {}</p>
<p><span class="label">Repartidor:</span> {} {}</p>
<p><span class="label">Vehiculo:</span> {} - {}</p>
<p><span class="label">Cliente emisor:</span> {}
    {}</p>
<p><span class="label">Receptor:</span> {}</p>
<p><span class="label">Kilometros:</span> {} km</p>
<p><span class="label">Peso:</span> {} kg</p>
<p><span class="label">Volumen:</span> {}</p>
</div>
</body>
</html>
" " "

```

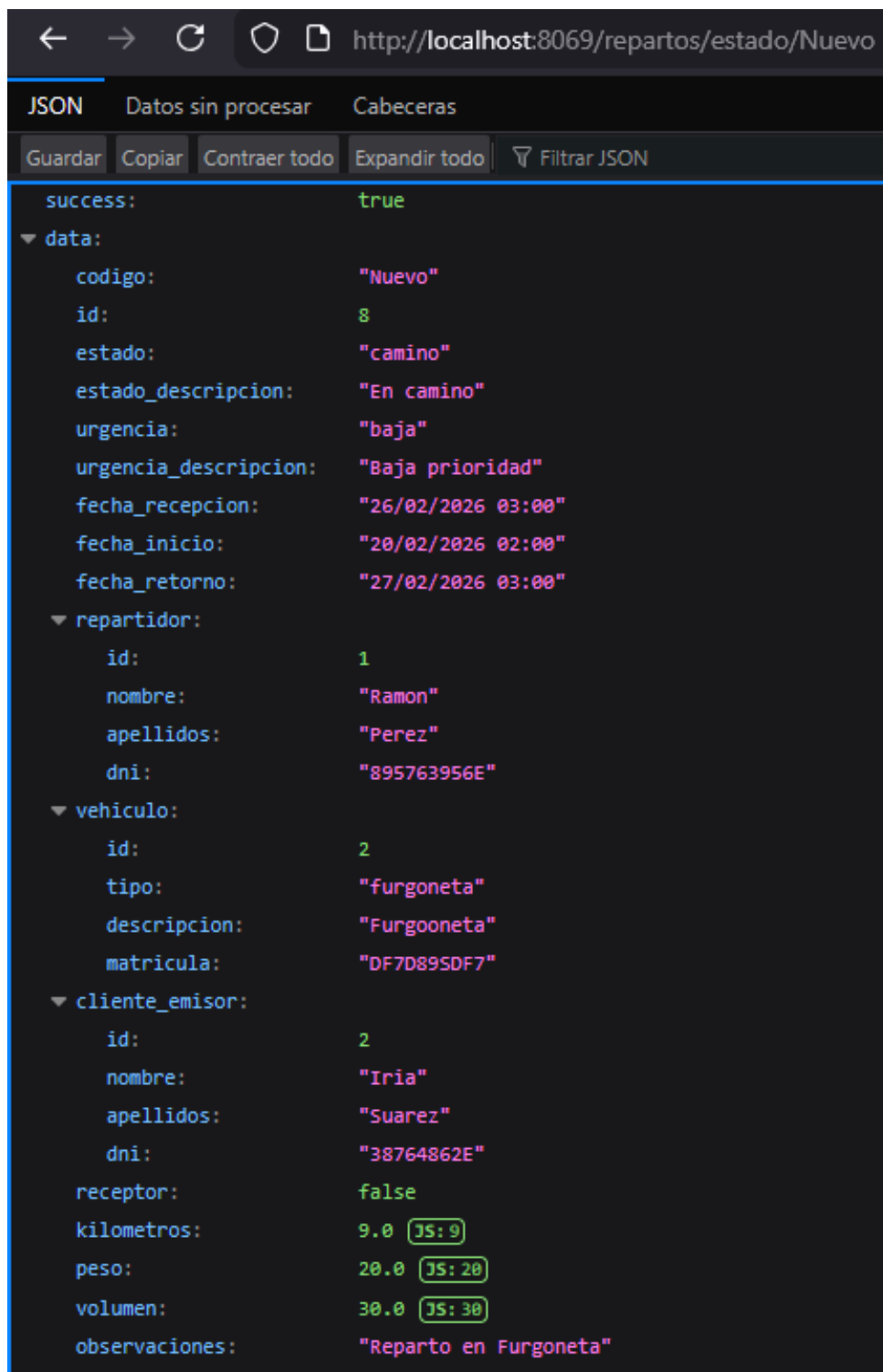
9. Ejemplos de uso del controlador web:

Test desde navegador, podemos ver que accede correctamente al controlador y muestra el estado del reparto:



Test desde elemento de prueba, aparece con codigo generico debido a que es una prueba

anterior:



El codigo lo genero mediante una api dentro de reparto. py:

```
@api.model
def create(self, vals):
    if vals.get('codigo', 'Nuevo') == 'Nuevo':
        vals['codigo'] = self.env['ir.sequence'].next_by_code('reparto.reparto') or 'Nuevo'
```



```
return super(Reparto, self).create(vals)
```

10. Security

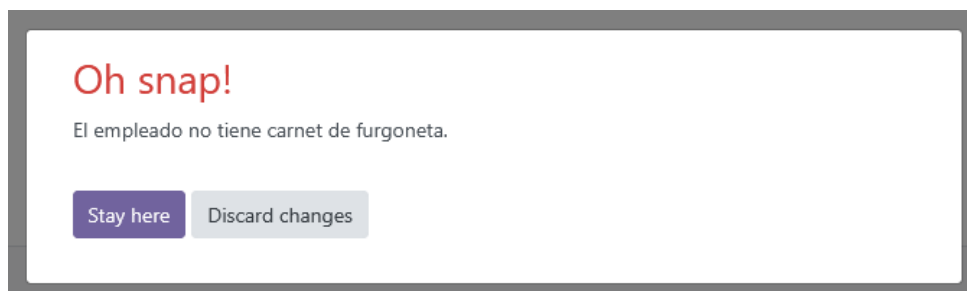
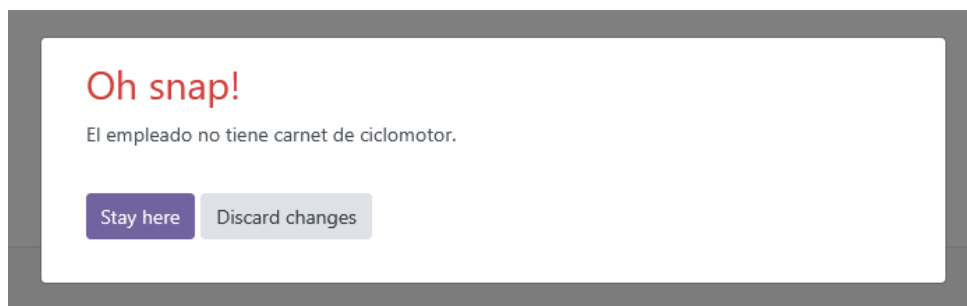
Aqui muestro el archivo de seguridad con los permisos basicos para cada modelo, simplemente doy acceso total como dice el ejercicio:

```
id ,name, model_id:id ,group_id:id ,perm_read ,perm_write ,perm_create ,
perm_unlink
access_reparto_empleado ,access_reparto_empleado ,
model_reparto_empleado ,base.group_user ,1,1,1,1
access_reparto_vehiculo ,access_reparto_vehiculo ,
model_reparto_vehiculo ,base.group_user ,1,1,1,1
access_reparto_cliente ,access_reparto_cliente ,
model_reparto_cliente ,base.group_user ,1,1,1,1
access_reparto_reparto ,access_reparto_reparto ,
model_reparto_reparto ,base.group_user ,1,1,1,1
```

11. Casos de uso:

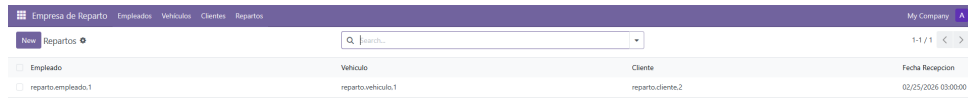
En este apartado mostraré capturas de pantalla de cada uno de los casos de uso pedidos en el enunciado, mostrando el error que se produce al no cumplir la restriccion.

11.1. Reparto empleado sin carnet adecuado:



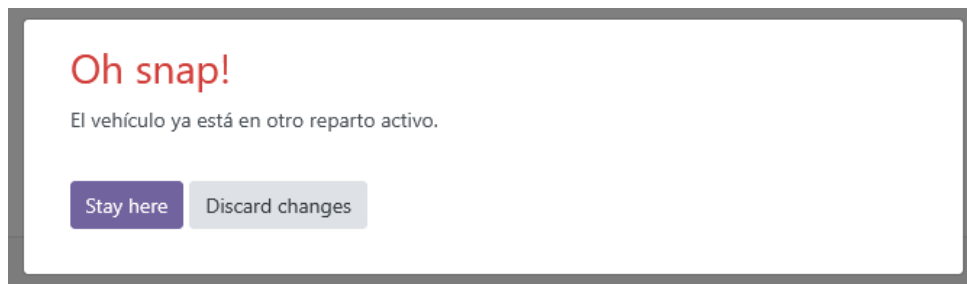
11.2. Reparto empleado con carnet adecuado:

Permite crearla correctamente sin mostrar ningun error:

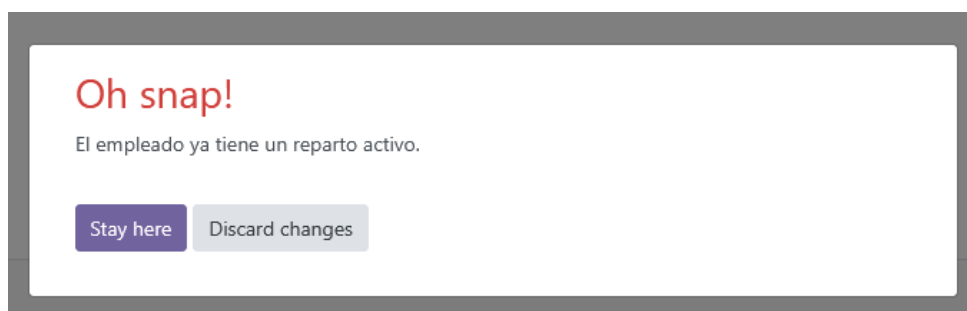


Empleado	Vehiculo	Cliente	Fecha Recpcion
reparto.empleado.1	reparto.vehiculo.1	reparto.cliente.2	02/25/2026 03:00:00

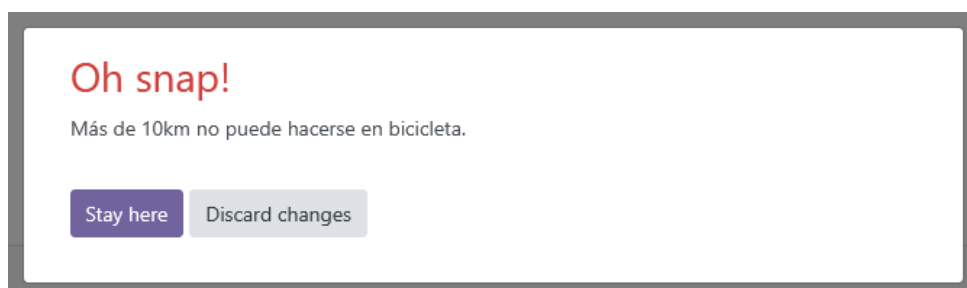
11.3. Reparto con vehiculo ocupado:



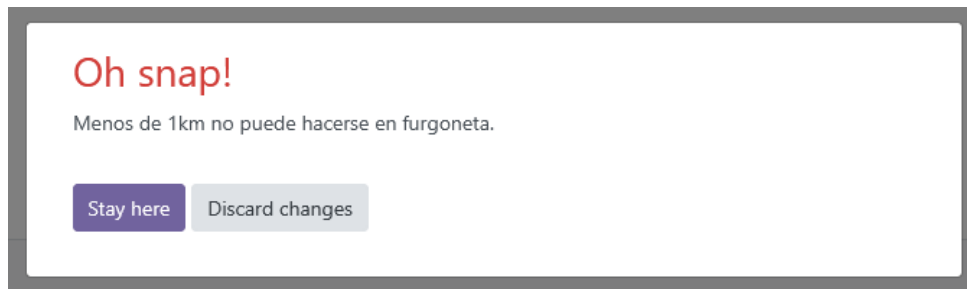
11.4. Reparto con empleado ya activo:



11.5. Reparto de mas de 10km en bicicleta:



11.6. Reparto de menos de 1km en furgoneta:



Con este tenemos el ejercicio completo con las restricciones adecuadas, el wizard y el informe. Me encontré con numerosos problemas los cuales me hicieron perder horas, principalmente debido a como está diseñado odoo, desde la lentitud para realizar cada prueba a los errores genéricos que normalmente no indican el problema real.